

Design Pattern

1. Adapter Pattern

The Adapter Pattern is used to make incompatible interfaces work together. In this project, two implementations can be identified: the AI vendor adapter and the Firebase authentication adapter.

1a. AI Vendor Adapter

File:

backend/server/src/services/ai/AiProvider.js

Target: AiProvider. SpeechService and VisionService depend on the common AiProvider interface. Adapter: OpenAiCompatibleProvider. It translates the application's chat/transcription calls into the HTTP API format expected by the configured AI vendor.

Relevant code:

```
export class AiProvider {
  get available() { return false; }

  async chat(_messages, _options) {
    throw new Error("AiProvider.chat() must
be implemented");
  }

  async transcribe(_audio) {
    throw new Error("AiProvider.transcribe()
must be implemented");
  }
}

export class OpenAiCompatibleProvider
extends AiProvider {
  constructor({
    apiKey,
    baseUrl,
    model,
    visionModel,
    transcribeModel,
```

```
    keyHeader
  }) {
    /* ... */
  }

  async chat(
    messages,
    { model, temperature = 0.3, json = false,
    maxTokens = 700 } = {}
  ) {
    const res = await fetch(
      `${this.baseUrl}/chat/completions`,
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          ...this.authHeaders
        },
        body: JSON.stringify({
          model: model || this.model,
          messages,
          temperature,
          max_tokens: maxTokens
        })
      }
    );

    const body = await res.json();

    return
    body.choices?.[0]?.message?.content ?? "";
  }
}

Why it is an Adapter: The application uses
the AiProvider interface, while
OpenAiCompatibleProvider converts those
calls into the external vendor's HTTP
request format. This keeps vendor-specific
details out of SpeechService and
VisionService.
```

1b. Firebase Authentication Adapter

File:

backend/server/src/services/auth/FirebaseAuthService.js

Target: The application's authentication service expects normalized user information such as sub, email, name, and picture.

Adaptee: firebase-admin SDK. Adapter: FirebaseAuthService. It calls Firebase's verifyIdToken() and translates the Firebase result into the application's expected format.

Relevant code:

```
export class FirebaseAuthService {
  constructor({
    adminAuth = buildDefaultAdminAuth()
  } = {}) {
    this.adminAuth = adminAuth;
  }

  async verify(idToken) {
    const decoded =
      await
this.adminAuth.verifyIdToken(idToken,
true);

    return {
      sub: String(decoded.uid),
      email,
      name: decoded.name,
      picture: decoded.picture
    };
  }
}
```

Why it is an Adapter:

AuthService.loginWithFirebase() only works with the normalized authentication data returned by FirebaseAuthService. It does not need to know the Firebase Admin SDK interface or its response structure.

2. Strategy Pattern

File:

backend/server/src/services/search/strategies.js and
backend/server/src/services/search/SearchService.js

The project uses separate interchangeable strategies for text, voice, and visual search.

Relevant code:

```
export class SearchStrategy {
  constructor(repository =
productRepository) {
    this.repository = repository;
  }

  async execute(_input) {
    throw new
Error("SearchStrategy.execute() must be
implemented");
  }
}
```

```
export class TextSearchStrategy extends
SearchStrategy {
  // text search implementation
}
```

```
export class VoiceSearchStrategy extends
SearchStrategy {
  // voice search implementation
}
```

```
export class VisualSearchStrategy extends
SearchStrategy {
  // visual search implementation
}
```

Context:

```
export class SearchService {
  constructor(strategies) {
    this.strategies =
strategies ??
new Map([
  ["text", new TextSearchStrategy()],
  ["voice", new VoiceSearchStrategy()],
]);
}
```

```

        ["visual", new VisualSearchStrategy()],
    ));
}

search(mode, input) {
    const strategy = this.strategies.get(mode);

    if (!strategy)
        throw badRequest(`Unknown search
mode: ${mode}`);

    return strategy.execute(input);
}

```

Why it is a Strategy Pattern: SearchService selects an interchangeable search algorithm according to the search mode while all strategies follow the common execute() operation.

3. Observer Pattern

File: frontend/src/lib/http/TokenStore.ts

The TokenStore maintains listeners that observe authentication-token changes. Subscribers are added through subscribe(), and set() notifies every registered listener when the token changes.

Relevant code:

```

private listeners =
    new Set<(token: string | null) => void>();

subscribe(
    listener: (token: string | null) => void
): () => void {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
}

```

```

set(token: string | null): void {
    this.token = token;
    this.listeners.forEach(l => l(token));
}

```

Why it is an Observer Pattern: TokenStore is the observable subject, the subscribed listeners are observers, and set() performs the notification when the observed token state changes.

4. Facade Pattern

File: frontend/src/lib/api/client.ts

The apiClient provides a simple entry point to several API repositories and the underlying HTTP system. Frontend components do not need to manage the individual repositories and HTTP details directly.

Relevant code:

```

export const apiClient = {
    auth: new AuthRepository(http),
    products: new ProductRepository(http),
    reviews: new ReviewRepository(http),
    cart: new CartRepository(http),
    orders: new OrderRepository(http),
    rewards: new RewardRepository(http),
    sellers: new SellerRepository(http),
    users: new UserRepository(http),
    ai: new AiRepository(http),
};

```

Why it is a Facade Pattern: apiClient hides the complexity of the HTTP client, token handling, and individual repositories behind one simple interface for frontend consumers.

Our project uses four of the design patterns covered in our course: Adapter, Strategy, Observer, and Facade. Adapter is used for integrating external services through

AuthProvider/OpenAiCompatibleProvider and FirebaseAuthService. Strategy is used for text, voice, and visual product search. Observer is used in TokenStore to notify subscribers when the authentication token changes. Facade is used through apiClient to provide a simple interface over the HTTP client and multiple API repositories. We did not identify clear implementations of Decorator or Builder, so we do not claim them.

Testing Code

1. Login / AuthService Test

File:
backend/server/tests/services/auth/AuthService.test.js

Purpose: Tests the application's login/authentication service. The example below uses mocked dependencies so the test focuses on AuthService behavior.

```
import { describe, it, expect, vi } from "vitest";
import { AuthService } from "../../src/services/auth/AuthService.js";

describe("AuthService", () => {
  it("logs in a user successfully", async () => {
    const firebaseAuth = {
      verify: vi.fn().mockResolvedValue({
        sub: "123",
        email: "user@example.com",
        name: "Test User",
        picture: null
      })
    };
    const users = {
      findOrCreateFromIdentity:
        vi.fn().mockResolvedValue({
          _id: "user-1",
          email: "user@example.com",
          name: "Test User"
        })
    };
    const tokens = {
      issue: vi.fn().mockResolvedValue({
```

```
    accessToken: "test-access-token"
  })
};

const service = new AuthService({
  firebaseAuth,
  users,
  tokens
});

const result =
  await service.loginWithFirebase("firebase-token");

expect(firebaseAuth.verify)
  .toHaveBeenCalledWith("firebase-token");

expect(users.findOrCreateFromIdentity)
  .toHaveBeenCalled();

expect(tokens.issue)
  .toHaveBeenCalled();

expect(result.accessToken)
  .toBe("test-access-token");
});
});
```

2. Firebase Authentication Test

File:
backend/server/tests/services/auth/FirebaseAuthService.test.js

Purpose: Tests the Firebase authentication adapter. The Firebase Admin SDK is mocked, and the test verifies that Firebase

data is converted into the application's expected identity format.

```
import { describe, it, expect, vi } from
"vitest";
import { FirebaseAuthService }
from
"../../src/services/auth/FirebaseAuthService.js";
```

```
describe("FirebaseAuthService", () => {
it("verifies a Firebase token and normalizes
the identity", async () => {
const adminAuth = {
verifyIdToken:
vi.fn().mockResolvedValue({
uid: "firebase-123",
email: "user@example.com",
name: "Test User",
picture: "profile.jpg"
})
});
```

```
const service =
new FirebaseAuthService({ adminAuth });
```

```
const result =
await service.verify("firebase-token");
```

```
expect(adminAuth.verifyIdToken)
.toHaveBeenCalledWith(
"firebase-token",
true
);
```

```
expect(result).toEqual({
sub: "firebase-123",
email: "user@example.com",
name: "Test User",
picture: "profile.jpg"
});
});
});
```

3. Search Service Test

File:
backend/server/tests/services/search/SearchS

ervice.test.js

Purpose: Tests the Strategy Pattern used by the search service. It verifies strategy selection, invalid modes, and registering a new strategy.

```
import { describe, it, expect, vi } from
"vitest";
import { SearchService }
from
"../../src/services/search/SearchService.js";
```

```
function fakeStrategy(result) {
return {
execute: vi.fn().mockResolvedValue(result)
};
}
```

```
describe("SearchService", () => {
it("dispatches to the registered strategy",
async () => {
const textStrategy =
fakeStrategy({ items: ["rice"] });
```

```
const service =
new SearchService(
new Map([["text", textStrategy]])
);
```

```
const result =
await service.search(
"text",
{ q: "rice" }
);
```

```
expect(textStrategy.execute)
.toHaveBeenCalledWith({ q: "rice" });
```

```
expect(result)
.toEqual({ items: ["rice"] });
});
```

```
it("rejects an unknown search mode", () => {
const service =
new SearchService(new Map());
```

```

expect(() =>
service.search("barcode", {}))
).toThrow(
"Unknown search mode: barcode"
);
});

it("allows a new strategy to be registered",
async () => {
const barcodeStrategy =
fakeStrategy({
items: ["barcode-product"]
});

const service =
new SearchService(new Map());

service.register(
"barcode",
barcodeStrategy
);

const result =
await service.search(
"barcode",
{ code: "123" }
);

expect(result).toEqual({
items: ["barcode-product"]
});
});
});

```

4. Recommendation Engine Test

File:

backend/server/tests/services/recommendation/RecommendationEngine.test.js

Purpose: Tests recommendation generation, user purchase history, and recommendation ordering. Repositories are mocked so the test does not require a real database.

```

import { describe, it, expect, vi } from
"vitest";
import { RecommendationEngine }

```

```

from
"../../src/services/recommendation/RecommendationEngine.js";

```

```

const catalogue = [
{
_id: "p1",
name: "Product 1",
category: "Rice",
rating: 4.5
},
{
_id: "p2",
name: "Product 2",
category: "Oil",
rating: 4.0
},
{
_id: "p3",
name: "Product 3",
category: "Rice",
rating: 4.8
}
];

```

```

function makeRepos({ orders = [] } = {}) {
return {
products: {
all: vi.fn()
.mockResolvedValue(catalogue)
},
orders: {
forUser: vi.fn()
.mockResolvedValue(orders)
}
};
}

```

```

describe("RecommendationEngine", () => {
it("returns recommendations for an
anonymous user", async () => {
const { products, orders } =
makeRepos();

```

```

const engine =
new RecommendationEngine({

```

```
products,  
orders,  
scorers: []  
});  
  
const result =  
await engine.recommend(null, 8);
```

```
expect(result.length)  
.toBeGreaterThan(0);
```

```
expect(products.all)  
.toHaveBeenCalled();  
});
```

```
it("excludes products already purchased by  
the user", async () => {  
  const { products, orders } =  
    makeRepos({  
      orders: [  
        {  
          items: [  
            { product: "p1" }  
          ]  
        }  
      ]  
    });
```

```
const engine =  
new RecommendationEngine({  
  products,  
  orders,  
  scorers: []  
});
```

```
const result =  
await engine.recommend(  
  { _id: "user-1" },  
  8  
);
```

```
expect(  
  result.find(  
    (product) => product._id === "p1"  
  )  
).toBeUndefined();
```

```
});
```

```
it("returns products in recommendation  
order", async () => {  
  const { products, orders } =  
    makeRepos();
```

```
const engine =  
new RecommendationEngine({  
  products,  
  orders,  
  scorers: []  
});
```

```
const result =  
await engine.recommend(null, 8);
```

```
const ids =  
result.map((product) => product._id);
```

```
expect(  
  ids.indexOf("p3")  
).toBeLessThan(  
  ids.indexOf("p2")  
);  
});  
});
```